

Hilfsmittel

Unicode-Masken

Unicode(hex)	Maske
bis 7F	0xxxxxxx
bis 7FF	110xxxxx 10xxxxxx
bis FFFF	1110xxxx 10xxxxxx 10xxxxxx
bis 10FFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Zeichen und die dezimalen ASCII-Werte

chr:	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
asc: 32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
chr: 0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
asc: 48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
chr: @	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
asc: 64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
chr: P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
asc: 80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
chr: `	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
asc: 96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
chr: p	q	r	s	t	u	v	w	x	y	z	{		}	~	
asc: 112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127

Weitere Zeichen mit ihrem dezimalen Unicode-Codepoint:

ä 228 - ö 246 - ü 252 - Ä 196 - Ö 214 - Ü 220 - ß 223 - € 8364
 alpha α 945 - beta β 946 - gamma γ 947 - delta δ 948 - epsilon ε 949
 pik ♠ 9824 - herz ♥ 9825 - karo ♦ 9826 - kreuz ♣ 9827

BOM für UTF-8 Codierung: EFBBBF

1. (2 Punkte) ANSI ist eine Erweiterung des ASCII-Codes auf 8 Bit. Im ANSI-Code hat 'a' die Codierung 01100001, 'A' die Codierung 01000001. Dekodiere die ANSI-Code Bitfolge:

01000100 01100001 01110010 01110100

Lösung: Dart

2. (3 Punkte) Das Zeichen für den Violschlüssel in der Musiknotation hat den hexadezimalen Unicode Code Point 1D11E. Gib die entsprechende UTF-8 Codierung in hexadezimaler Schreibweise an.

Lösung:

```
Gegebener Codepoint : 1D11E
Binärdarstellung : 0001 1101 0001 0001 1110
Maske : 1111 0xxx 10xx xxxx 10xx xxxx 10xx xxxx
eingesetzt : 1111 0000 1001 1101 1000 0100 1001 1110
UTF-Codierung : F09D849E
```

3. (3 Punkte) Ein Texteditor stellt folgende Bitfolge dar:

```
1100 0011 1001 1111 1110 0000 1010 0101 1000 0000 0011 1111
```

Gib die hexadezimalen Codepoints der dargestellten Zeichen an. Prüfe, ob die Zeichen in der Hilfsmitteltabelle auftauchen und gib dann an, um welche Zeichen es sich handelt.

Lösung:

```
1100 0011 1001 1111 1110 0000 1010 0101 1000 0000 0011 1111
```

Die Bitfolge enthält 3 Zeichen:

Zeichen 1: codepoint dezimal = 223, codepoint hex = df, Zeichen ß

Zeichen 2: codepoint dezimal = 2368, codepoint hex = 940, Zeichen nicht in Hilfstabelle

Zeichen 3: codepoint dezimal = 63, codepoint hex = 3f, Zeichen ?

4. (3 Punkte) a. Was sagt die Fano-Bedingung aus?

b. Unten sind Codewörter für einige Buchstaben gegeben. Ist die Fano-Bedingung erfüllt (Begründe)? Finde ein Beispiel für eine Codierung, die nicht eindeutig dekodiert werden kann.

a = 00 b = 11 c = 01 d = 10 e = 101 f = 110

Lösung: a. Die Fanobedingung besagt, dass kein Codewort der Anfang eines anderen Codewortes sein darf. Wenn die Fanobedingung erfüllt ist, lassen sich Codierungen eindeutig decodieren.

b. Das Codewort für b ist der Anfang des Codeworts für f. Die Fano-Bedingung ist nicht erfüllt. Beispiel für eine nicht eindeutige Codierung: 110110101 kann mit bcde oder fbcc dekodiert werden.

5. (3 Punkte) Decodiere die Bitfolge mit dem angegebenen Huffman-Baum. Gib für jedes Zeichen die Codierung an.

```
1110001001111000101111001011010011010
```

Hinweis zur Darstellung des Huffman-Baums: Der Baum ergibt sich durch Drehen der abgebildeten Zeilen um 90 Grad nach rechts. Die Punkte geben die Ebenen des Baumes an. Knoten, die nur Zahlen aber keine Zeichen enthalten, sind mit einem # markiert.

```
...1 v
...2 #
...1 b
..4 #
...2 t
.8 #
..4 r
14 #
...2 a
..3 #
...1 g
.6 #
..3 e
```

Lösung:

b->1110, e->00, r->10, a->011, t->110, v->1111, g->010

Kodiertes Wort: beratervertrag

6. (3 Punkte) Konstruiere nach dem Verfahren aus dem Unterricht einen Huffman-Baum für das Wort **gegengerade**. Gib für jeden Buchstaben die Codierung an.

Lösung:

Huffmanbaum für: **gegengerade**

```
..4 e
.7 #
..3 g
11 #
...1 d
..2 #
...1 a
.4 #
...1 r
..2 #
...1 n
```

Codierung:

```
g -> 10
e -> 11
n -> 000
r -> 001
a -> 010
d -> 011
```

7. (3 Punkte) a. Benenne im folgenden Code alle Beispiele für die folgenden Begriffe: globale Variable, lokale Variable, Parameter und Argument.
b. Was wird ausgegeben?

```
x = 6
def calc(y):
    a = 9
    return a*y

z = calc(4)+x
print(z)
```

Lösung:

- a. x, z sind globale Variablen, a, y sind lokale Variable, y ist Parameter, 4 ist Argument.
b. 42

8. (1 Punkt) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
returns: string, die Zeichenfolge 'abc'

>>> los()
'abc'
'''
```

Lösung:

```
def los():
    return 'abc'
```

9. (2 Punkte) Schreibe eine Funktion, die sich so verhält, wie im folgenden docstring beschrieben.

```
'''
s string, k >= 0, k optional mit default 1
returns: string, die ersten k zeichen von s

>>> make('Hallo ')
'H'
>>> make('Hallo ',3)
'Hal'
'''
```

Lösung:

```
def make(s, k=1):
    return s[:k]
```

10. (2 Punkte) Was erscheint auf der Konsole?

```
def a(x,y):
    return x + 2*y
def b(x):
    return x - 1
def c(x,y):
    return x - y
def d(x,y):
    return x - y > 0
z = c(a(3,4),b(2))
print(d(z,12),z)
```

Lösung: False 10

11. (1 Punkt) Was wird ausgegeben?

```
x = 1000
y = 1000
print(x is y, x == y)
```

Lösung: False True

12. (2 Punkte) Übersetze den folgenden Pseudocode in Python-Code:

```
Falls der Variablen best kein Wert zugeordnet ist oder x größer als best ist:  
    setze best auf x
```

Lösung:

```
if best is None or x > best:  
    best = x
```

13. (1 Punkt) Was wird ausgegeben?

```
c = [12]  
d = c  
d.append(8)  
c.append(4)  
print(c, d)
```

Lösung:

```
[12, 8, 4] [12, 8, 4]
```

14. (2 Punkte) Was wird ausgegeben?

```
def doit(x):  
    x.append(12)  
    a = [7]  
    return x  
  
a = [4]  
b = doit(a)  
print(a, b)
```

Lösung:

```
[4, 12] [4, 12]
```

15. (3 Punkte) Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid und den Rückgabewert wenn die binäre Suche auf die Liste a und das gesuchte Element x angewendet wird.

```
def binaereSuche(a, x):
    L = 0
    R = len(a)-1
    while L <= R:
        mid = (L+R)//2
        if a[mid] == x:
            return mid
        if a[mid] < x:
            L = mid + 1
        else:
            R = mid - 1
    return -1

x = 10
a = [4, 12, 17, 21, 23, 27, 28, 29, 31, 34, 40, 42]
```

Lösung:

x = 10

	0	1	2	3	4	5	6	7	8	9	10	11
	4	12	17	21	23	27	28	29	31	34	40	42

Schleife	L	R	mid	return
0	0	11		
1	0	4	5	
2	0	1	2	
3	1	1	0	
4	1	0	1	-1

16. (3 Punkte) Die rekursive Funktion `sterneDazwischen` fügt zwischen jedem Zeichen des übergebenen Strings das Zeichen '*' ein. Strings, die aus weniger als 2 Zeichen bestehen, bleiben unverändert. Implementiere diese Funktion.

```
'''
>>> sterneDazwischen('lol')
'l*o*l'

>>> sterneDazwischen('a')
'a'
'''
```

Lösung:

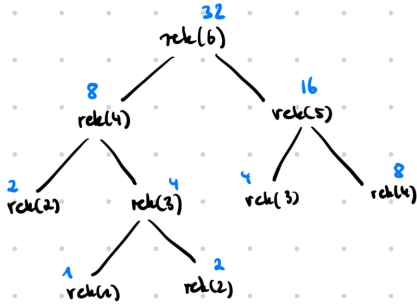
```
def sterneDazwischen(s):
    if len(s) <= 1:
        return s
    return s[0]+'*'+sterneDazwischen(s[1:])
```

17. (3 Punkte) Gib einen geeigneten Rekursionsbaum zur Ermittlung der Zwischenergebnisse an. Was erscheint auf der Konsole?

```
def rek(n):
    if n <= 2: return n
    return 2*rek(n-2) + rek(n-1)

print(rek(6))
```

Lösung:



Die Ausgabe ist 32.

18. (3 Punkte) Gib die Abfolge der rekursiven Aufrufe an. Was erscheint auf der Konsole?

```
def rek(a,b):
    if b == 0: return a+1
    return rek(2*b+2,a%b)

print(rek(15,20))
```

Lösung:

```
rek(15,20)
rek(42,15)
rek(32,12)
rek(26,8)
rek(18,2)
rek(6,0)
Die Ausgabe ist 7
```

19. (2 Punkte) Die rekursive Funktion hanoi wird mit hanoi(3,'a','c','b') aufgerufen. Wie heißen der 8. und 9. Aufruf von hanoi?

```
def hanoi(n, start, ziel, zwischen):
    if n == 0: return
    hanoi(n-1, start, zwischen, ziel)
    print("Scheibe",n,"von",start,"nach",ziel)
    hanoi(n-1, zwischen, ziel, start)
```

Lösung:

```
8 hanoi(0,'a','b','c')
9 hanoi(2,'b','c','a')
```

Aufgabe:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	Summe:
Punkte:	2	3	3	3	3	3	3	1	2	2	1	2	1	2	3	3	3	3	2	45